

JubileeOutlook PostgreSQL Offline Cache

Step-by-Step Execution Plan

Version: 1.0
Date: January 15, 2026
Project: JubileeOutlook WPF Client
Estimated Phases: 5

Executive Summary

This execution plan outlines the implementation of PostgreSQL-based offline caching for the JubileeOutlook WPF email client. The implementation enables users to access emails, calendar events, and contacts without network connectivity, with automatic synchronization when connectivity is restored.

Phase 1: Project Setup & Dependencies

Step 1.1: Add NuGet Packages

Action: Add PostgreSQL packages to JubileeOutlook.csproj

```
cd d:\Data\JubileeEnterprise.com\applications\JubileeOutlook.wpf\JubileeOutlook
dotnet add package Npgsql --version 8.0.1
dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL --version 8.0.0
```

Verification:

- [] Packages appear in JubileeOutlook.csproj
- [] dotnet restore completes successfully
- [] No version conflicts

Step 1.2: Update Configuration

Action: Add PostgreSQL connection settings to appsettings.json

File: JubileeOutlook/appsettings.json

```
{
  "LocalCache": {
    "ConnectionString": "Host=localhost;Port=5432;Database=jubilee_outlook_cache;Username=jubilee_user;Password=your_password",
    "EnableOfflineMode": true,
    "SyncIntervalSeconds": 300,
    "MaxCacheAgeDays": 30
  }
}
```

Verification:

- [] Configuration file updated
- [] Connection string is valid
- [] Settings are accessible via ConfigurationService

Step 1.3: Create Configuration Model

Action: Create LocalCacheSettings.cs

File: JubileeOutlook/Models/LocalCacheSettings.cs

```
namespace JubileeOutlook.Models;

public class LocalCacheSettings
{
    public string ConnectionString { get; set; } = string.Empty;
    public bool EnableOfflineMode { get; set; } = true;
    public int SyncIntervalSeconds { get; set; } = 300;
    public int MaxCacheAgeDays { get; set; } = 30;
}
```

Verification:

- [] Model created
- [] Properties match appsettings.json structure

Phase 2: Database Schema Implementation

Step 2.1: Install PostgreSQL Locally

Action: Ensure PostgreSQL is installed and running

For Windows:

```
# Check if PostgreSQL is installed
pg_isready -h localhost -p 5432

# If not installed, download from:
# https://www.postgresql.org/download/windows/
```

Verification:

- ☐ PostgreSQL service is running
- ☐ Can connect via psql or pgAdmin

Step 2.2: Create Database

Action: Create the local cache database

```
-- Connect as postgres superuser
CREATE DATABASE jubilee_outlook_cache;
CREATE USER jubilee_user WITH PASSWORD 'your_secure_password';
GRANT ALL PRIVILEGES ON DATABASE jubilee_outlook_cache TO jubilee_user;
```

Verification:

- ☐ Database created
- ☐ User has proper permissions

Step 2.3: Create Core Tables

Action: Execute the following SQL script

File: Scripts/create_cache_schema.sql

```
-- Enable UUID extension
CREATE EXTENSION IF NOT EXISTS "uuid-oss";

-- =====
-- CACHED EMAILS TABLE
-- =====
CREATE TABLE IF NOT EXISTS cached_emails (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    server_id VARCHAR(255) UNIQUE NOT NULL,
    folder_id VARCHAR(255) NOT NULL,
    subject TEXT,
    sender_name VARCHAR(255),
    sender_email VARCHAR(255),
    recipients JSONB DEFAULT '[]',
    cc_recipients JSONB DEFAULT '[]',
    bcc_recipients JSONB DEFAULT '[]',
    body TEXT,
    body_preview VARCHAR(500),
    is_html BOOLEAN DEFAULT FALSE,
    is_read BOOLEAN DEFAULT FALSE,
    is_flagged BOOLEAN DEFAULT FALSE,
    is_draft BOOLEAN DEFAULT FALSE,
    has_attachments BOOLEAN DEFAULT FALSE,
    attachments JSONB DEFAULT '[]',
    importance VARCHAR(20) DEFAULT 'normal',
    received_date TIMESTAMPTZ,
    sent_date TIMESTAMPTZ,
    conversation_id VARCHAR(255),
    cached_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    last_modified TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    is_deleted BOOLEAN DEFAULT FALSE,
    sync_status VARCHAR(20) DEFAULT 'synced'
);

-- Create indexes for performance
CREATE INDEX idx_emails_folder ON cached_emails(folder_id);
CREATE INDEX idx_emails_received ON cached_emails(received_date DESC);
CREATE INDEX idx_emails_sender ON cached_emails(sender_email);
CREATE INDEX idx_emails_sync ON cached_emails(sync_status);
CREATE INDEX idx_emails_search ON cached_emails USING GIN(to_tsvector('english', subject || ' ' || COALESCE(body, '')));

-- =====
-- CACHED FOLDERS TABLE
-- =====
CREATE TABLE IF NOT EXISTS cached_folders (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    server_id VARCHAR(255) UNIQUE NOT NULL,
    name VARCHAR(255) NOT NULL,
    folder_type VARCHAR(50),
    parent_folder_id VARCHAR(255),
    unread_count INTEGER DEFAULT 0,
    total_count INTEGER DEFAULT 0,
    display_order INTEGER DEFAULT 0,
    cached_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    last_modified TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);
```

```

CREATE INDEX idx_folders_type ON cached_folders(folder_type);
CREATE INDEX idx_folders_parent ON cached_folders(parent_folder_id);

-- =====
-- CACHED CALENDAR EVENTS TABLE
-- =====
CREATE TABLE IF NOT EXISTS cached_events (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    server_id VARCHAR(255) UNIQUE NOT NULL,
    calendar_id VARCHAR(255),
    title VARCHAR(500) NOT NULL,
    description TEXT,
    location VARCHAR(500),
    start_time TIMESTAMPTZ NOT NULL,
    end_time TIMESTAMPTZ NOT NULL,
    is_all_day BOOLEAN DEFAULT FALSE,
    is_recurring BOOLEAN DEFAULT FALSE,
    recurrence_pattern JSONB,
    organizer_name VARCHAR(255),
    organizer_email VARCHAR(255),
    attendees JSONB DEFAULT '[]',
    status VARCHAR(50) DEFAULT 'confirmed',
    reminder_minutes INTEGER,
    is_private BOOLEAN DEFAULT FALSE,
    categories JSONB DEFAULT '[]',
    cached_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    last_modified TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    is_deleted BOOLEAN DEFAULT FALSE,
    sync_status VARCHAR(20) DEFAULT 'synced'
);

CREATE INDEX idx_events_calendar ON cached_events(calendar_id);
CREATE INDEX idx_events_start ON cached_events(start_time);
CREATE INDEX idx_events_end ON cached_events(end_time);
CREATE INDEX idx_events_sync ON cached_events(sync_status);

-- =====
-- CACHED CONTACTS TABLE
-- =====
CREATE TABLE IF NOT EXISTS cached_contacts (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    server_id VARCHAR(255) UNIQUE NOT NULL,
    display_name VARCHAR(255),
    first_name VARCHAR(100),
    last_name VARCHAR(100),
    email_addresses JSONB DEFAULT '[]',
    phone_numbers JSONB DEFAULT '[]',
    company VARCHAR(255),
    job_title VARCHAR(255),
    department VARCHAR(255),
    notes TEXT,
    photo_url VARCHAR(500),
    cached_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    last_modified TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    is_deleted BOOLEAN DEFAULT FALSE,
    sync_status VARCHAR(20) DEFAULT 'synced'
);

CREATE INDEX idx_contacts_name ON cached_contacts(display_name);
CREATE INDEX idx_contacts_email ON cached_contacts USING GIN(email_addresses);
CREATE INDEX idx_contacts_sync ON cached_contacts(sync_status);

-- =====
-- SYNC QUEUE TABLE
-- =====
CREATE TABLE IF NOT EXISTS sync_queue (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    entity_type VARCHAR(50) NOT NULL,
    entity_id VARCHAR(255) NOT NULL,
    operation VARCHAR(20) NOT NULL,
    payload JSONB,
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    retry_count INTEGER DEFAULT 0,
    last_error TEXT,
    status VARCHAR(20) DEFAULT 'pending'
);

CREATE INDEX idx_queue_status ON sync_queue(status);
CREATE INDEX idx_queue_created ON sync_queue(created_at);
CREATE INDEX idx_queue_entity ON sync_queue(entity_type, entity_id);

-- =====
-- SYNC STATE TABLE
-- =====
CREATE TABLE IF NOT EXISTS sync_state (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

```

```

    entity_type VARCHAR(50) UNIQUE NOT NULL,
    last_sync_token VARCHAR(500),
    last_sync_time TIMESTAMPTZ,
    full_sync_required BOOLEAN DEFAULT TRUE
);

-- Insert default sync states
INSERT INTO sync_state (entity_type, full_sync_required)
VALUES
    ('emails', TRUE),
    ('folders', TRUE),
    ('events', TRUE),
    ('contacts', TRUE)
ON CONFLICT (entity_type) DO NOTHING;

```

Verification:

- [] All tables created successfully
- [] Indexes created
- [] Default sync states inserted

Phase 3: Service Implementation

Step 3.1: Create LocalCacheService

Action: Implement the local cache service

File: JubileeOutlook/Services/LocalCacheService.cs

Implementation Tasks:

1. [] Create class with Npgsql connection management
2. [] Implement connection string handling from configuration
3. [] Add InitializeDatabaseAsync() method
4. [] Implement email caching methods:
 - [] CacheEmailAsync(EmailMessage email)
 - [] GetCachedEmailsAsync(string folderId)
 - [] GetCachedEmailByIdAsync(string serverId)
 - [] UpdateEmailStatusAsync(string serverId, bool isRead, bool isFlagged)
 - [] MarkEmailDeletedAsync(string serverId)
5. [] Implement folder caching methods:
 - [] CacheFolderAsync(MailFolder folder)
 - [] GetCachedFoldersAsync()
6. [] Implement event caching methods:
 - [] CacheEventAsync(CalendarEvent event)
 - [] GetCachedEventsAsync(DateTime start, DateTime end)
7. [] Implement contact caching methods:
 - [] CacheContactAsync(Contact contact)
 - [] GetCachedContactsAsync()
 - [] SearchContactsAsync(string query)

Verification:

- [] Service compiles without errors
- [] Connection pooling works correctly
- [] CRUD operations function properly

Step 3.2: Create SyncQueueService

Action: Implement offline operation queue

File: JubileeOutlook/Services/SyncQueueService.cs

Implementation Tasks:

1. [] Create QueueOperationAsync(string entityType, string entityId, string operation, object payload)
2. [] Create GetPendingOperationsAsync()
3. [] Create MarkOperationCompletedAsync(Guid operationId)
4. [] Create MarkOperationFailedAsync(Guid operationId, string error)
5. [] Create ClearCompletedOperationsAsync()

Verification:

- [] Operations are queued correctly
- [] Queue retrieval works
- [] Status updates function properly

Step 3.3: Create NetworkStatusService

Action: Implement network connectivity detection

File: JubileeOutlook/Services/NetworkStatusService.cs

Implementation Tasks:

1. ☐ Implement `IsOnline` property
2. ☐ Implement `NetworkStatusChanged` event
3. ☐ Add API health check endpoint monitoring
4. ☐ Implement automatic status polling (every 30 seconds)
5. ☐ Handle network interface changes

Verification:

- ☐ Online/offline detection works
- ☐ Events fire correctly on status change
- ☐ API health check functions properly

Step 3.4: Create SyncService

Action: Implement background synchronization

File: `JubileeOutlook/Services/SyncService.cs`

Implementation Tasks:

1. ☐ Create background sync timer
2. ☐ Implement `ProcessSyncQueueAsync()`
3. ☐ Implement `PullChangesFromServerAsync()`
4. ☐ Implement `PushChangesToServerAsync()`
5. ☐ Implement conflict resolution logic
6. ☐ Handle sync state management (delta sync tokens)
7. ☐ Implement full sync fallback

Verification:

- ☐ Background sync runs on schedule
 - ☐ Queue operations are processed
 - ☐ Server changes are pulled correctly
 - ☐ Conflicts are resolved appropriately
-

Phase 4: Integration with Existing Services**Step 4.1: Update ApiMailService**

Action: Integrate cache with mail service

File: `JubileeOutlook/Services/ApiMailService.cs`

Modifications:

1. ☐ Add `LocalCacheService` dependency
2. ☐ Add `NetworkStatusService` dependency
3. ☐ Modify `GetMessagesAsync()`:
 - Check network status
 - If offline, return cached data
 - If online, fetch from API and update cache
4. ☐ Modify `SendMessageAsync()`:
 - If offline, queue operation
 - If online, send immediately
5. ☐ Modify `DeleteMessageAsync()`:
 - Update local cache
 - If offline, queue operation
6. ☐ Modify `MoveMessageAsync()`:
 - Update local cache
 - If offline, queue operation

Verification:

- ☐ Service works in offline mode
- ☐ Cache is updated on API calls
- ☐ Operations are queued when offline

Step 4.2: Update ApiCalendarService

Action: Integrate cache with calendar service

File: `JubileeOutlook/Services/ApiCalendarService.cs`

Modifications:

1. ☐ Add `LocalCacheService` dependency
2. ☐ Add `NetworkStatusService` dependency
3. ☐ Implement offline-aware event fetching
4. ☐ Implement offline event creation queuing

Verification:

- ☐ Calendar works offline

- ☐ Events are cached properly

Step 4.3: Update UI for Offline Status

Action: Add visual offline indicators

Files:

- MainWindow.xaml
- MainWindow.xaml.cs

Tasks:

1. ☐ Add offline status indicator in status bar
2. ☐ Add "Sync pending" badge for queued operations
3. ☐ Show sync progress during synchronization
4. ☐ Disable certain actions when offline (if needed)

Verification:

- ☐ Offline status is clearly visible
- ☐ Pending operations count shows correctly
- ☐ Sync progress is displayed

Phase 5: Testing & Validation

Step 5.1: Unit Testing

Action: Create unit tests for cache services

File: JubileeOutlook.Tests/Services/LocalCacheServiceTests.cs

Test Cases:

1. ☐ Test email caching and retrieval
2. ☐ Test folder caching
3. ☐ Test event caching
4. ☐ Test contact caching
5. ☐ Test sync queue operations

Step 5.2: Integration Testing

Action: Test end-to-end scenarios

Test Scenarios:

1. ☐ Start application offline - verify cached data loads
2. ☐ Go offline while using app - verify queuing works
3. ☐ Come back online - verify sync completes
4. ☐ Conflict resolution - modify same item offline/online
5. ☐ Large data sync - verify performance

Step 5.3: Performance Testing

Action: Validate cache performance

Metrics to Measure:

1. ☐ Email list load time (cached vs API)
2. ☐ Search performance on cached data
3. ☐ Memory usage with large cache
4. ☐ Sync queue processing speed

Deliverables Checklist

New Files to Create

File	Phase Status	
Models/LocalCacheSettings.cs	1	☐
Scripts/create_cache_schema.sql	2	☐
Services/LocalCacheService.cs	3	☐
Services/SyncQueueService.cs	3	☐
Services/NetworkStatusService.cs	3	☐
Services/SyncService.cs	3	☐

Files to Modify

File	Phase Status	
JubileeOutlook.csproj	1	☐
appsettings.json	1	☐
Services/ApiMailService.cs	4	☐
Services/ApiCalendarService.cs	4	☐
MainWindow.xaml	4	☐
MainWindow.xaml.cs	4	☐

Risk Mitigation

Potential Issues & Solutions

Risk	Impact	Mitigation
PostgreSQL not installed	High	Provide installer script or embedded option
Connection string exposed	Medium	Use secure credential storage
Large cache size	Medium	Implement cache expiration/cleanup
Sync conflicts	Medium	Implement clear conflict resolution rules
Network detection failure	Low	Multiple detection methods (API + system)

Timeline Reference

Phase	Description	Dependencies
Phase 1	Project Setup	None
Phase 2	Database Schema	Phase 1
Phase 3	Service Implementation	Phase 2
Phase 4	Integration	Phase 3
Phase 5	Testing	Phase 4

Quick Reference Commands

Database Commands

```
# Check PostgreSQL status
pg_isready -h localhost -p 5432

# Connect to database
psql -h localhost -U jubilee_user -d jubilee_outlook_cache

# Run schema script
psql -h localhost -U jubilee_user -d jubilee_outlook_cache -f Scripts/create_cache_schema.sql
```

.NET Commands

```
# Add packages
dotnet add package Npgsql --version 8.0.1

# Build project
dotnet build

# Run tests
dotnet test
```

Support Resources

- **PostgreSQL Documentation:** <https://www.postgresql.org/docs/>
- **Npgsql Documentation:** <https://www.npgsql.org/doc/>
- **InspireContinuum API:** Internal API documentation
- **Project Repository:** GitHub repository for JubileeOutlook

Document End

This execution plan should be followed sequentially. Each phase builds upon the previous one. Mark checkboxes as tasks are completed to track progress.